

AT2 Computer Vision Task

Dr. Christopher Özbek - 26239780

Abstract

Multi-class image classification was a challenging computer vision task, which up to a few years ago, attracted a lot of research attention and saw considerable progress. In this assignment task, we were asked to perform transfer learning and finetuning on important models from the class of convolutional neural networks (CNN) to recognize food categories from the Food101 dataset (Bossard, Guillaumin, & Van Gool, 2014). First, GoogLeNet, ResNet-50, and MobileNetV3 were used as pre-trained starting point models and their classification heads were replaced and retrained on the Food101 dataset over 15 epochs. MobileNetV3 yielded the highest baseline accuracy of 58.45% and was then finetuned for another 15 epochs by unfreezing specific layers. The final configuration, which unfroze the 'features.15' block and utilized a reduced learning rate of 0.00005, achieved a final test accuracy of 67.38% lagging state-of-the-art finetunes achieving over 90% accuracy.

1 Introduction

Visual recognition of food items is a difficult machine learning task due to the high intra-class variance and inter-class similarity of food items. The Food101 dataset was originally presented in 2014 to demonstrate advances in using the machine learning algorithm Random Forests for classification tasks and in which the authors achieved an average accuracy of 50.76% (Bossard, Guillaumin, & Van Gool, 2014). The authors already noted back then that their models were being outclassed by CNNs.

1.1 Data presentation

The Food101 dataset consists of 101,000 images representing 101 distinct food classes. The dataset is balanced providing 1,000 images per class. The images originate from the 101 most popular and consistently named foods published on foodspotting.com (defunct in 2026) and were subsequently split into 750 training and 250 test images per category. Test images were manually cleaned and reviewed by Bossard et al. All images are 3 channel RGB images, where the longest image dimension is 512 pixels. The images are cropped to feature just the food item, plate, bowl, etc. and table background. The images were taken in real world situations such as restaurants and show non-professional composition, presentation and lighting. Image quality is in line with pre-2014 mobile phone and photo cameras.

The original split 750 training and 250 testing images was read from the dataset meta description. Then the 750 training images were further split into 85% training samples and 15% validation samples to guide hyper-parameter tuning and model selection and still retain an unseen testing set.

My initial experiments showed that training a single epoch of the models would take up to 8 minutes, which would have consumed all Google Colab resources available to me. I thus made the decision to optimize data throughput by preprocessing. All images were uniformly resized upfront to have a shortest edge of 256 pixels while maintaining their aspect ratio. Using the

PyTorch DataLoader pipeline, images were randomly cropped to 224 by 224 pixels to match the native training data sizes of the three models to evaluate. Data augmentation was applied exclusively to the training set to improve model generalization and consisted of flipping and 90 degree random rotation using PyTorch transforms.v2 on the GPU. Finally, all images were normalized using standard ImageNet mean and standard deviation values.

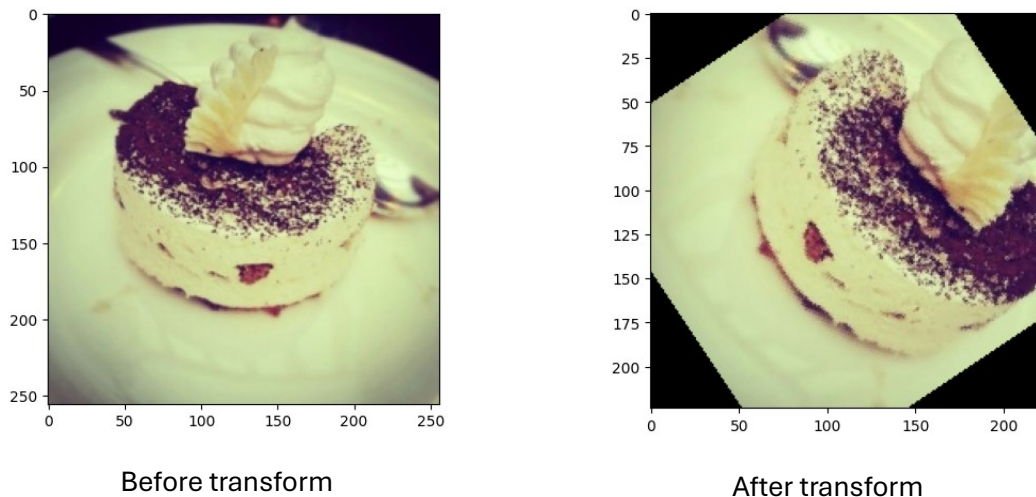


Figure 1: Exemplary food image (label 'Tiramisu') from the Food101 dataset before and after data augmentation transform (flip and rotate).

1.2 Neural Network Architectures and Hyperparameters

In line with the assignment brief, I utilized transfer learning using weights pre-trained on the ImageNet dataset. Three important historical network architectures were evaluated to establish a baseline:

- GoogLeNet (Szegedy et al., 2015), utilizing multi-scale Inception modules, i.e. applying multiple convolutional kernel sizes in parallel and then using a dimensionality reduction step to bring the parameter count back down before continuing in a similar manner in sequence. An important evolutionary advancement of the CNN class of models (75,000 citations).
- ResNet-50 (He et al., 2016), utilizing residual skip connections to address vanishing gradients. The key insight being to have later neural network layers learn only the residual difference between the previous layer's output and the target, rather than learning the full mapping. One of the most-cited papers in the development towards large-scale neural network architectures (330,000 citations).
- MobileNetV3 (Howard et al., 2019), a hardware-optimized architecture introducing several engineering advances such as depthwise separable convolutions, using inverted residual blocks that expand to a higher-dimensional channel space for richer feature extraction before compressing back down, and employing a hardware-optimized activation function (hard-swish). Published near the end of the CNN-dominated era as attention-based architectures were beginning to take over (15,000 citations).

Hyperparameter Selection: Due to the strict computational limits of the Google Colab T4 free tier, training a single 15-epoch configuration took approximately one hour and consumed a significant portion of my daily compute quota. Consequently, an exhaustive grid search over the hyperparameter space was impossible. Instead, I relied on default heuristics and lab exercises (So 2026) as a starting point.

The Adam optimizer was used with an initial learning rate of 0.001 (I experimented also with 0.0001 as described in the technical appendix). For the custom classification head replacing the ImageNet defaults, I implemented a 'funnel' design using fully connected layers (1024 -> 512 -> 101 neurons). It was my hope that a gradual focusing of the neurons would preserve information until the last decision making layer. I used a dropout of 20% following our lab notebooks. A separate experiment with dropout of 10%+5% between layers did yield higher accuracy, but also more hints of overfitting with only marginal gains in final accuracy (see technical appendix). I used early stopping after 3 epochs and an LR plateau scheduler, but due to the low number of epochs this didn't have a relevant impact on outcomes.

2 Part A: Transfer Learning Experiments

In the exploratory phase, the original classification heads of all three models were removed. The feature extraction backbones were frozen and only my custom 1024->512->101 head was trained. Training was capped at 15 epochs and was performed on the shuffled training dataset. Early stopping and LR reduction was set to a patience of 3 epochs. After training was completed, test set accuracy was determined.

Results

GoogLeNet performed the poorest considering both final validation loss and test set accuracy. The network triggered early stopping at epoch 13 after validation loss failed to improve, plateauing at a test accuracy of 44.27%. Since GoogLeNet is the oldest of the three architectures, this result is not very surprising.

ResNet-50 showed moderate improvement, completing the full 15 epochs and reaching a test accuracy of 53.42%. ResNet-50 notable showed the strongest continued learning progress at epoch 15 and could potentially overtake MobileNetV3 given enough epochs. It was clearly the slowest model to train with roughly 3 minutes per epoch.

MobileNetV3 achieved the highest baseline performance, finishing with a test loss of 1.5946 and a test accuracy of 58.45%. It also trained the fastest with less than 2 minutes per epoch. MobileNetV3 was thus selected as the foundational model for Part B.

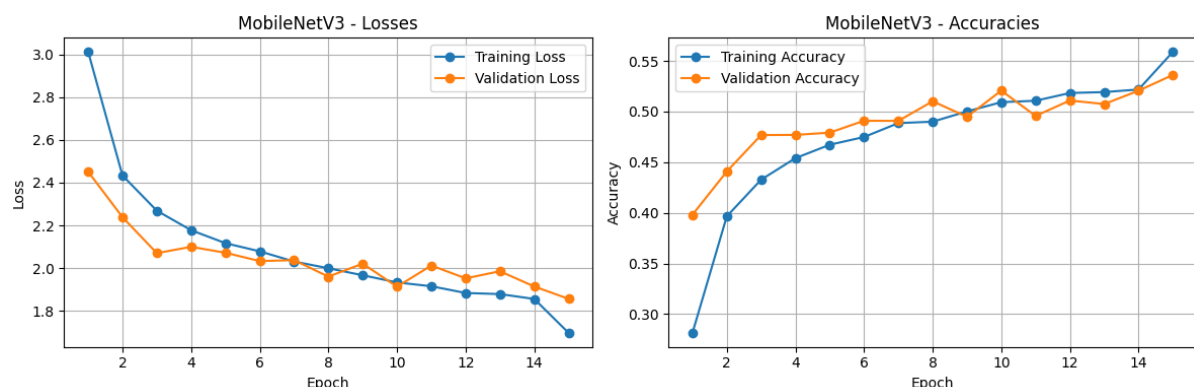


Figure 2: MobileNetV3 training chart for 15 epochs using LR=0.001 and 20% dropout. All convolutional layers were frozen, only the final classification head was allowed to train.

3 Part B: Fine-Tuning Experiments

Following the transfer learning phase, I performed four fine-tuning experiments. The objective was to improve MobileNetV3 by unfreezing selected layers and evaluating the outcome.

In each experiment, the model was initialized with the best weights saved from Part A. The entire network was frozen, and then a single specific layer block within the MobileNetV3 backbone was unfrozen. A lower learning rate was chosen (0.00005) to prevent destroying the existing weights after higher rates showed deteriorating performance with training. Out of the 17 layers (0 to 16), I chose 2 layers at the end (layer 15 and 16) and 2 layers in the middle (12 and 11) as I hypothesized that the earlier layers would be rather generic image understanding layers (e.g. colors/shapes) and the middle and later layers would do semantic feature work.

Results:

Unfreezing mid-level feature layer 11 and 12 yielded test accuracies of 64.17% and 66.32% respectively over the 58.4% baseline. Unfreezing the final backbone layer (16) resulted in a performance of 64.0%. Unfreezing features.15 provided the best overall result of the entire assignment, achieving a test accuracy of 67.38%. It seems that layer 15 is deep enough to have high level semantic features, but is not yet too focused on the final prediction as layer 16.



Figure 3: Training performance of fine-tuning on feature 15. Training and validation accuracy was still improving after 15 epochs pointing to benefits of additional training.

4 Error Analysis

After training, the model was stored as a PyTorch loadable weights file and the resulting model was explored in cursory fashion. To understand the limitations of the trained MobileNetV3 model, the top mis-classified categories were enumerated:

Actual Class	Predicted Class	Count
Tiramisu	Chocolate Cake	42
Tuna Tartare	Beef Tartare	40
Ramen	Pho	39
Bruschetta	Caprese Salad	39
Hamburger	Pulled Pork Sandwich	35
Pork Chop	Steak	34

Steak	Filet Mignon	32
Grilled Cheese Sandwich	Club Sandwich	32
Filet Mignon	Steak	31
Steak	Baby Back Ribs	31

Table 1: Top-misclassified images against the 250 images per class test set.

These top-misclassified categories all point to visually similar items and give us good confidence that the model isn't randomly misclassifying but is just struggling with categories which have very similar visual features or require careful comparison of many global attributes rather than color/texture/size (e.g. Steak vs. Filet Mignon steak). Exemplary mispredictions are given in the Figure below.

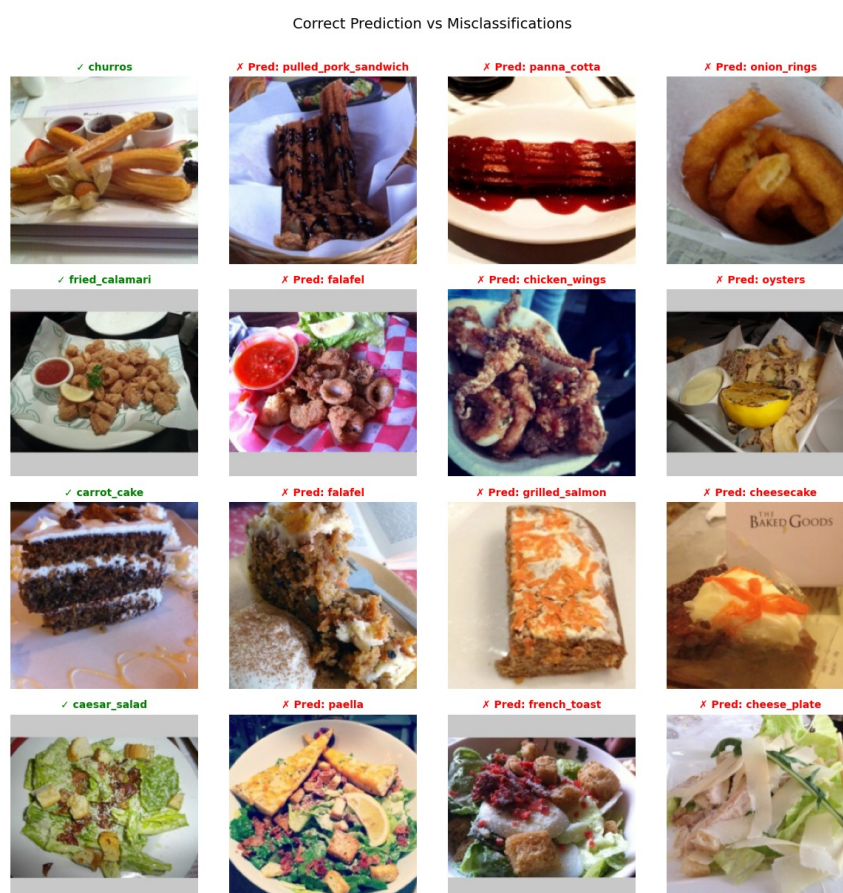


Figure 4: Exemplary misclassifications by our model

5 Discussion of Model Performance and Limitations

Baseline performance using a frozen MobileNetV3 backbone achieved 58.5% accuracy. By isolating and unfreezing layer 15, overall accuracy improved 9% to 67.4%. While this falls short of the 90%+ accuracies achieved by state-of-the-art models on this dataset, it demonstrates a clear and successful application of transfer learning and fine-tuning under the given constraints of the assignment.

Online search after performing my experiments revealed that up 90% accuracy is reported attainable with ResNet50 with the following adjustments (Jin 2019): Replacing the AvgPool2d

between convolutional layers and final classification head with a AdaptiveConcatPool2d with Avg and Max pooling side-by-side, using adaptive learning rate adjustment, unfreezing the entire model, training at various sizes, adding BatchNorm, increasing dropout to 25%+50% rather than decreasing it and various other small improvements. I tried to replicate some of the improvements but couldn't achieve substantial improvements (see technical appendix).

5.1 Limitations

These experiments have several limitations:

- **Compute/Time Constraints:** All experiments were forcibly capped at 15 epochs. The validation loss trajectories in the fine-tuning phase (Figure 2) suggest that the models were still slowly learning and had not yet fully converged. Overfitting would have to be carefully monitored.
- **Limited exploration of hyper-parameters:** We didn't fully explore the potential for adjustment in network sizes, layer-norms, activation function, dropouts and learning rates.
- **The dataset label quality wasn't analyzed.** If wrongly labeled images remain in the dataset, our results could be distorted. cursory examination of examples point to good data quality.
- **Limited data augmentation:** To stay within compute budgets, additional data augmentation techniques such as adding color jitter, perspective distortions, noise, etc. were not explored.
- **Fine-Tuning was limited to a single layer:** Unfreezing additional layers likely would have provided additional gains in accuracy.

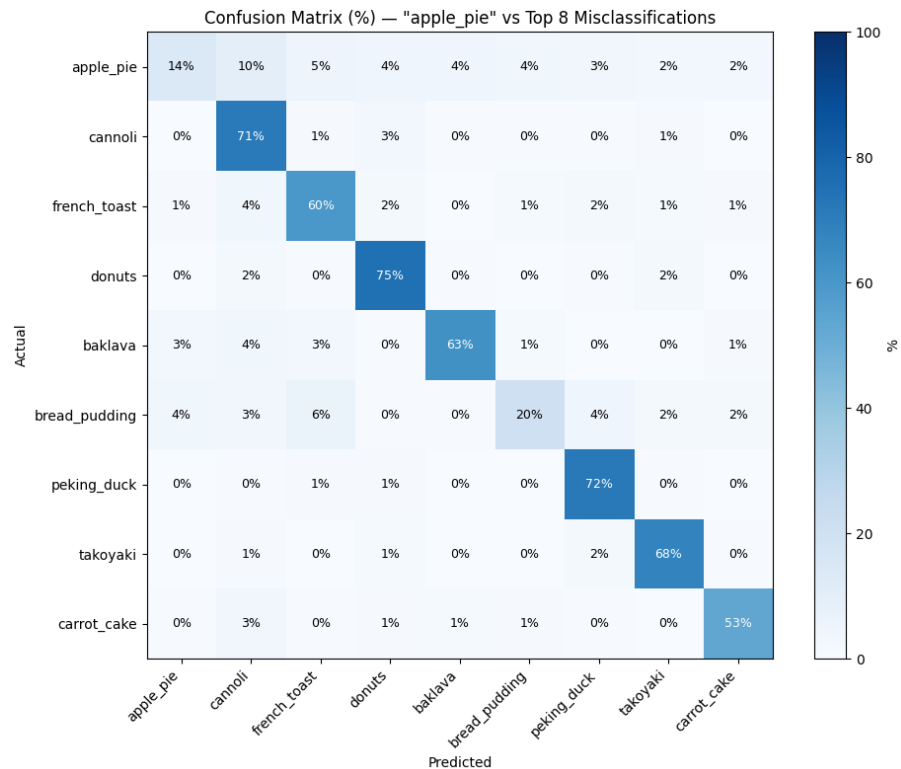


Figure 5: Condensed confusion matrix of the worst performing category (apple_pie) against the main misclassification categories.

5.2 Remaining Issues and Recommendations

- The resulting model performance shows still many gaps for distinguishing popular food categories, so putting the model in production must be carefully considered against the target categories and against state-of-the-art multi-modal LLM-based classification.
- Ethical considerations related to food classification weren't explored in depth. Since the dataset was balanced, there shouldn't be a bias between the individual class included in the dataset. Yet, the dataset in general seems largely focused on food available in Western/developed nations and doesn't represent diverse global food taste or include more than just the most famous dishes per non-western country if at all. It should be considered widening the dataset to capture tastes more widely. Socio-economic factors could also silently influence results, for instance via quality of picture taken by higher priced camera vs. models commonly found in poorer demographics or quality of food carriers such as nice ceramic plates vs. paper food containers.
- To reach state-of-the-art level of performance, it would be recommended to use newer model backbones first and perform more rigorous hyper-parameter exploration (e.g. learning rate finding) before performing the training in a more comprehensive manner than the explored 15 epochs. Additional unfreezing of more than one layer could also achieve additional performance gains.
- Increasing training dataset size beyond the 637 images available for training per class and incorporating more diverse images (e.g. more modern pictures) will be necessary to deploy a food classification model into production. Images currently are all dated before 2014 and thus no longer representative of modern food presentation, changes in global taste and other changes in popularity (e.g. plates, cutlery).

Bibliography & Acknowledgements

All code was written with the support of Google Gemini Flash 2.5 (Colab), Claude Code Opus 4.6 and Google Gemini 3.1 Pro in particular charting and data loading code.

- Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining Discriminative Components with Random Forests. In European Conference on Computer Vision (ECCV).
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770-778).
- Howard, A., Sandler, M., Chu, G., Chen, L. C., Chen, B., Tan, M et al. (2019). Searching for MobileNetV3. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 1314-1324).
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D et al. (2015). Going deeper with convolutions. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 1-9).
- W. So, A. So, et al. (2026), UTS Deep Learning, Autumn 2026. In particular laboratory notebooks for modules on CNN and transfer learning.
- Tao Jin (2019) Food 101 Image Classification Challenge Problem, Github Commit 61b5ebe, published https://github.com/Pylligent/food101-image-classification/blob/master/food-101_EDA_Model_RestNet_final.ipynb, accessed 02.04.2026

Technical Appendix

I ran the following additional, exploratory experiments which aren't included in my assignment submission (G = GoogLeNet, M = MobileNetV3, R = ResNet50)

Experiment Focus	Configuration Notes	Baseline	Outcome	Observation / Result
Different learning rate	Learning rate reduced 10-fold to 0.0001 Epochs 15 Batch 128 Dropout 20% LR Patient 3	G: 44.27% M: 58.45% R: 53.42%	G: 44.68% M: 55.67% R: 54.24%	MobileNetV3 as best baseline model didn't benefit from LR reduction. ResNet50 improved 1%.
Reduced dropout in FC head	Instead of 20% dropout between first/second and second/third layer used 10% + 5%. Epochs 15 Batch 128 LR Patient 3	G: 44.27% M: 58.45% R: 53.42%	G: 44.88% M: 59.09% R: 56.33%	Reduced dropout slightly improved overall model performance, but train/validation gap points to risk of overfitting. Since the gains are marginal, I wouldn't recommend this. Jin (2019) found better results with much increased dropout and increased learning rate.
Replicate FC from Jin 2019	Instead of 20% dropout between first/second and second/third layer used 25% + 50%. BatchNorm1D LR start at 0.01 instead of 0.001 Epochs 20 Batch 128 LR Patience 1	G: 44.27% M: 58.45% R: 53.42%	G: 47.79% M: 60.11% R: 58.15%	Results are improved but at the cost of additional training length. In particular ResNet50 profits and closes the gap to MobileNetV3 significantly. Jin reported reaching 74% within 7 epochs using fastai fit_one_cyle, which was unattainable for me with >4 times the number of epochs